Corso di Architettura e Progetto dei Calcolatori

Docente
Prof. Nicola Mazzocca

Studenti
Dott. Alberto Stravato
matr. M63001735
Dott. Emanuele Santacroce
matr. M63001716
Dott. Francesco Floriano
matr. M63001765

Anno Accademico 2024/25

L'IDEA

Ti sei mai chiesto come un pilota di F1 alleni i propri riflessi per essere sempre pronto in gara?
Nel nostro progetto, due giocatori si sfidano in un test di rapidità ispirato proprio a quello che i piloti fanno per attivare e disattivare il DRS nel minor tempo possibile.
Vince chi riesce a premere per primo, mettendo alla prova la propria velocità di reazione, proprio come in pista!

LE FIGURE DI GIOCO

A. Master: è il **cervello centrale** del sistema che orchestra l'intero gioco di riflessi tra due giocatori.
B. Giocatore 1: è il **protagonista attivo** del sistema che compete attraverso riflessi e velocità di reazione per determinare l'esito della sfida.
C. Giocatore 2: è il **protagonista attivo** del sistema che compete attraverso riflessi e velocità di reazione per determinare l'esito della sfida.

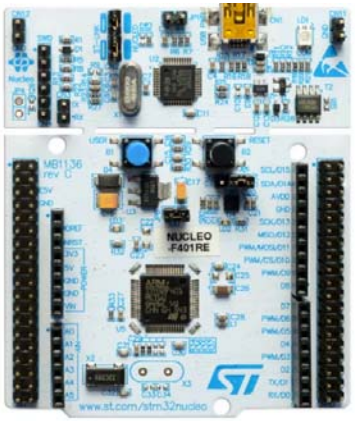
MASTER

Il Master coordina l'intero sistema di gioco gestendo la comunicazione con i due giocatori, controllando il timing attraverso delay casuali (1-10 secondi) per evitare anticipazioni, e orchestrando il flusso di gioco tramite stati sequenziali.
Raccoglie i tempi di reazione inviati dai giocatori, li confronta per determinare il vincitore con precisione al millisecondo, **gestisce situazioni di timeout** quando i giocatori non reagiscono entro 5 secondi, e **rileva partenze false** quando i pulsanti vengono premuti prematuramente.
Fornisce feedback centralizzato mostrando risultati dettagliati, countdown visivi durante l'attesa, e **coordina reset automatici** per nuove partite mantenendo sincronizzati tutti i dispositivi del sistema.

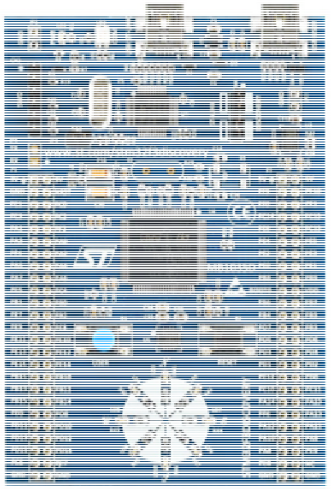
GIOCATORI 1-2

Il Giocatore gestisce l'esperienza di gioco individuale fornendo interfaccia completa con pulsante, feedback visivi/sonori personalizzati, e display dedicato che mostra stati specifici (attesa, pronto, vai, premuto, risultato).
Calcola tempi di reazione precisi misurando l'intervallo tra ricezione del segnale e pressione del pulsante, **mantiene sincronizzazione temporale** con il master, e **gestisce reset automatico** delle proprie variabili ad ogni nuova partita.
Fornisce feedback immediato multi-modale e **auto-gestisce il buzzer** con timeout automatico per evitare suoni prolungati, garantendo un'esperienza di gioco fluida e professionale per ogni singolo partecipante.

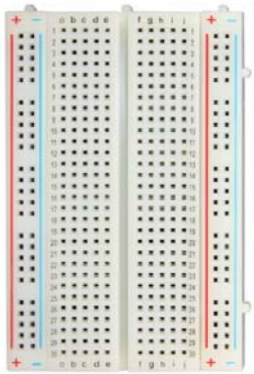
COSA ABBIAMO USATO



1x STM32F401- RE
– Nucleo 64



2x STM32F303
- Discovery



3x Breadboard

3x Modulo OLED
0.96 Pollici

1x Resistenza 330 Ohm

1x LED Verde

2x Buzzer Piezoelettrici Attivi

Jumpers
M-M
F-M

BUS E PROTOCOLLI DI COMUNICAZIONE

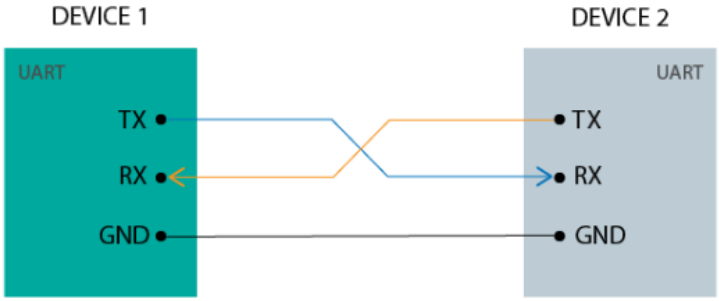
Le soluzioni adottate si basano sull'uso combinato e complementare dei protocolli **UART** e **I2C**, scelti appositamente per le loro caratteristiche distintive, perfettamente adatte ai diversi ruoli richiesti dall'architettura distribuita del sistema. In particolare:

A. UART: rappresenta la rete neurale nel nostro sistema, in quanto costituisce l'infrastruttura di comunicazione principale del sistema. **Il Master dialoga simultaneamente** con entrambi i giocatori attraverso due canali UART dedicati.
B. I2C: rappresenta gli occhi del nostro sistema. Ogni partecipante ha il proprio **display personale** che racconta la sua storia: dall'attesa del "READY!" all'esplosione di gioia del "HAI VINTO!".

UART

Per la comunicazione inter-nodo, sono stati utilizzati i dispositivi **USART** integrati sulle dev board, configurati in modalità **UART** (Universal Asynchronous Receiver/Transmitter).
In particolare, la configurazione scelta è la **8BN1S**.
Nel sistema sono presenti due canali di comunicazione bidirezionali che collegano rispettivamente il Master a ciascun Giocatore, attraverso i quali vengono scambiati:

A. Comandi da parte del Master che è il coordinatore del gioco.
B. I tempi di reazione da parte dei Giocatori.



BUS I2C

I²C è un sistema di comunicazione seriale sincrono sviluppato e brevettato da Philips nel 1982, diventato standard libero nel 2006 e da allora ampiamente diffuso nell'industria elettronica.
Il protocollo si basa su un'architettura multi-master che distingue tra device Master, responsabili dell'inizializzazione delle comunicazioni, e device Slave**.**
La comunicazione I2C richiede due linee seriali:

A. SDA (Serial DAta): linea dati.
B. SCL (Serial CLock): linea clock.

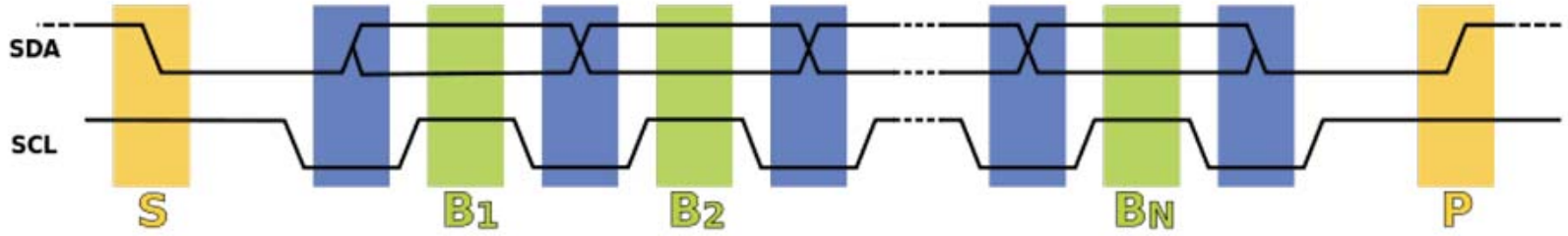
A queste vanno aggiunte altre due linee di collegamento ugualmente importanti, quali:

A. Vcc: le linee SDA e SCL vi sono connesse mediante resistori di pull-up.
B. Ground: definisce un riferimento comune per tutti i devices connessi.

Le caratteristiche **fondamentali** dell'I²C sono che lo stato di riposo delle linee SDA e SCL corrisponde al valore logico alto.
Inoltre, per supportare la possibilità di ospitare più device master, il protocollo implementa un sistema di arbitraggio del bus che consente ai master di pilotare le linee solo quando devono iniziare una comunicazione.

BUS I2C:TEMPIFICAZIONE DEI SEGNALI

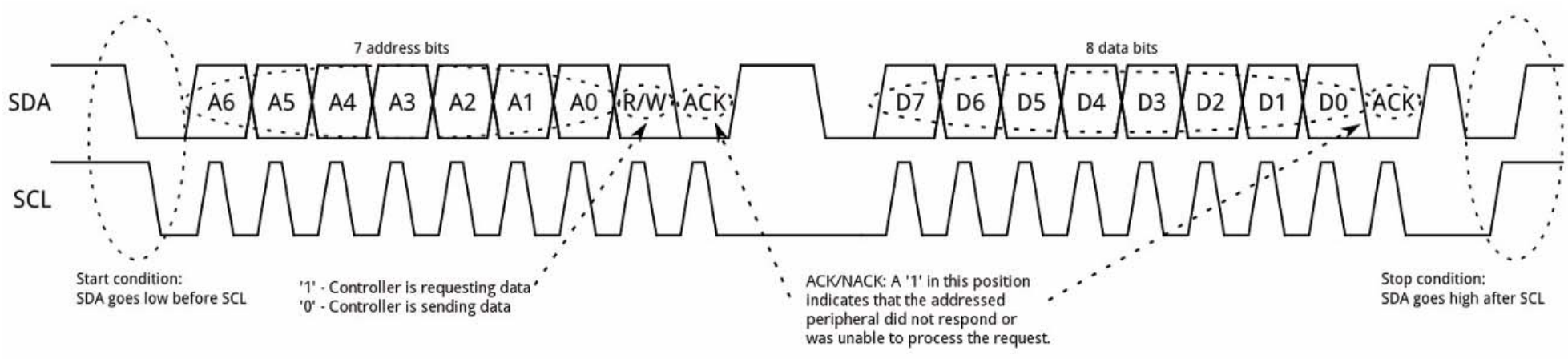
La comunicazione **I²C** segue una sequenza temporale precisa che garantisce la sincronizzazione tra master e slave. La transazione inizia con lo **START** bit, generato dal master forzando la linea SDA da livello alto a livello basso mentre il clock SCL rimane alto, segnalando a tutti i dispositivi l'inizio di una nuova comunicazione.
Durante la **trasmissione dati**, il master prima porta SCL a livello logico basso, poi impone sulla linea SDA il valore del bit da trasmettere.
La commutazione di SCL da basso ad alto indica che il dato è **stabile** e può essere letto in sicurezza dai dispositivi slave.
La transazione si conclude con lo **STOP** bit, generato commutando SDA da livello basso ad alto mentre SCL è mantenuto alto.



BUS I2C:TRANSAZIONI I

Il protocollo I²C definisce diversi tipi fondamentali di transazioni, ognuna delle quali inizia con uno **START** bit e termina con uno **STOP** bit.
La transazione di tipo messaggio singolo rappresenta la forma più elementare di comunicazione e può essere di due tipologie: scrittura di dati da un master verso uno slave, oppure lettura di dati da parte del master da uno slave specifico.
La sequenza di comunicazione inizia quando il master genera lo **START** bit e procede inviando l'**indirizzo** del dispositivo slave con cui intende comunicare, utilizzando i bit B1 fino a B7.
Immediatamente dopo l'indirizzo, il master trasmette il bit di **controllo W/R** (Write/Read) che specifica la direzione della transazione: se intende trasferire informazioni verso lo slave o ricevere informazioni da esso.
Il meccanismo di **acknowledgment** garantisce l'affidabilità della comunicazione: se lo slave indirizzato è presente e disponibile sul bus, prende il controllo della linea SDA durante il successivo impulso alto di SCL e la forza a livello basso, generando il segnale **ACK**. Questo conferma al master che il dispositivo selezionato ha correttamente ricevuto la richiesta ed è pronto a rispondere, stabilendo così una comunicazione affidabile tra i due dispositivi.

BUS I2C:TRANSAZIONI 2



I2C: ARBITRAGGIO DEL BUS

Il protocollo I²C implementa una politica di arbitraggio deterministica che consente a più dispositivi master di coesistere sullo stesso bus senza generare conflitti.
Il meccanismo si basa sul principio che ogni trasmettitore monitora costantemente il livello della linea dati SDA e lo confronta con il valore che intende trasmettere.
L'arbitraggio sfrutta la natura **open-drain** delle linee I²C: quando un dispositivo imposta SDA a livello logico alto (1) e contemporaneamente un altro dispositivo la forza a livello basso (0), il risultato fisico sulla linea sarà sempre il livello basso.
Il primo trasmettitore, osservando che il livello effettivo della linea è diverso da quello previsto, riconosce immediatamente che un altro nodo sta trasmettendo simultaneamente.
Il dispositivo che per primo rileva questa discrepanza perde automaticamente l'arbitrato e abbandona l'interazione corrente, cessando di pilotare la linea SDA.

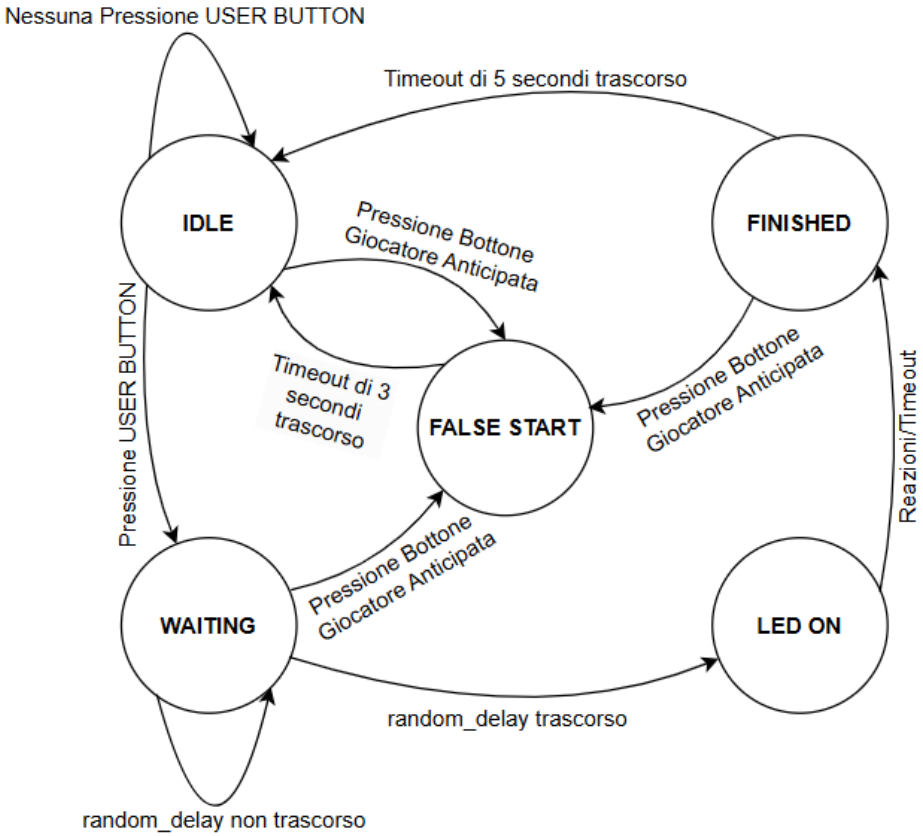
MASTER: MACCHINA A STATI I

La **macchina a stati** costituisce il paradigma di controllo centrale dell'intero sistema, con il **Master** incaricato di orchestrare il flusso del gioco attraverso una sequenza ordinata e deterministica di stati.
Ogni stato rappresenta una fase ben definita del ciclo operativo (ad esempio: menu, countdown, acquisizione delle reazioni, visualizzazione dei risultati), e le transizioni tra stati sono attivate da **eventi temporali** o da **input esterni**, come segnali dai Giocatori.
Il sistema prevede **cinque stati principali**:

- **IDLE**: fase di attesa per l'avvio di una nuova partita.
- **WAITING**: gestione del countdown casuale prima del segnale di inizio.
- **LED_ON**: fase attiva in cui vengono raccolti i tempi di reazione.
- **FINISHED**: elaborazione e presentazione dei risultati.
- **FALSE_START**: rilevamento e gestione degli errori dovuti a partenze anticipate.

Questa architettura a stati finiti assicura un comportamento **predicibile, modulare e facilmente estendibile**, dove ogni stato ha **responsabilità ben definite** e le condizioni di transizione sono chiaramente specificate, rendendo l'intero sistema affidabile e di semplice manutenzione.

MASTER: MACCHINA A STATI 2



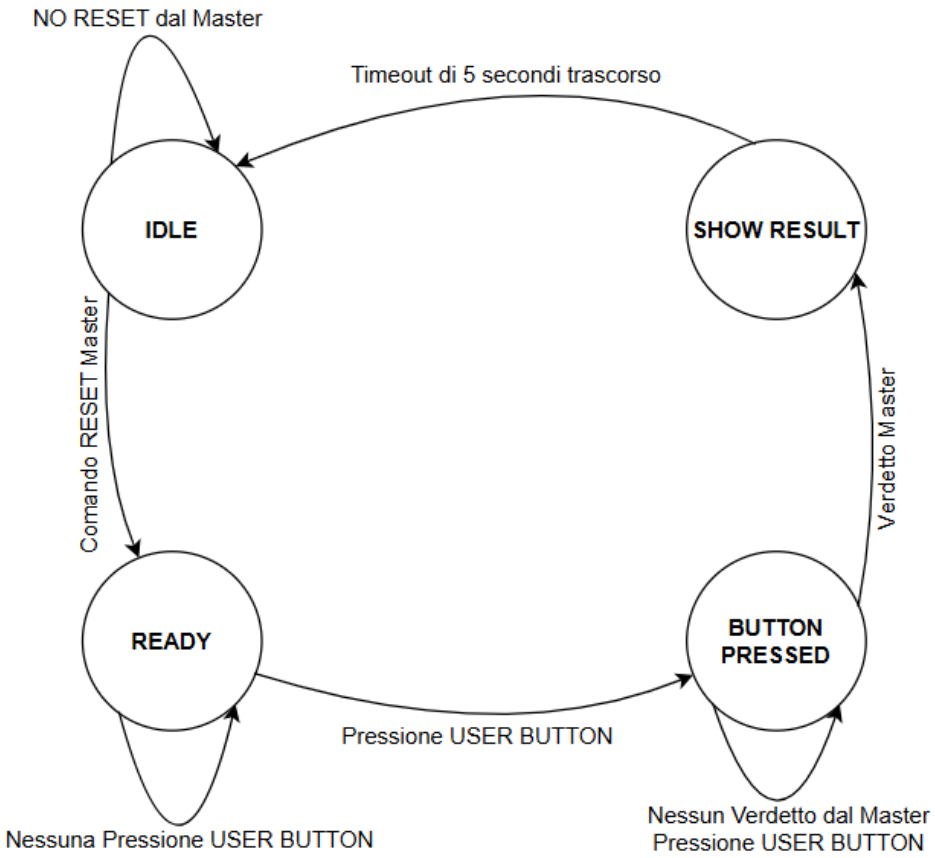
GIOCATORE 1-2: MACCHINA A STATI 1

La macchina a stati dello **Slave** gestisce il comportamento di ciascun **Player Controller**, garantendo una risposta reattiva e sincronizzata sia ai comandi del **Master** che agli input locali del giocatore.
Ogni stato rappresenta una fase dell'interazione utente, dalla semplice attesa fino alla rilevazione precisa del tempo di reazione.
Le transizioni sono attivate principalmente da **comandi UART** ricevuti dal Master e da eventi locali, come la **pressione del pulsante**.
Il sistema include **quattro stati principali**:

- **IDLE**: attesa dell'avvio partita tramite comando RESET dal Master.
- **READY**: fase di attesa del segnale visivo (GO), con rilevazione immediate della pressione del pulsante e invio del tempo di reazione.
- **BUTTON_PRESSED**: attesa del rilascio del pulsante e del verdetto (WIN/LOSE)..
- **SHOW_RESULT**: visualizzazione locale dell'esito ricevuto.

Questa architettura reattiva e distribuita consente una **misurazione accurata e puntuale dei tempi di reazione**, mantenendo la **sincronizzazione con il Master** e offrendo un'interfaccia fluida e indipendente per ciascun giocatore.

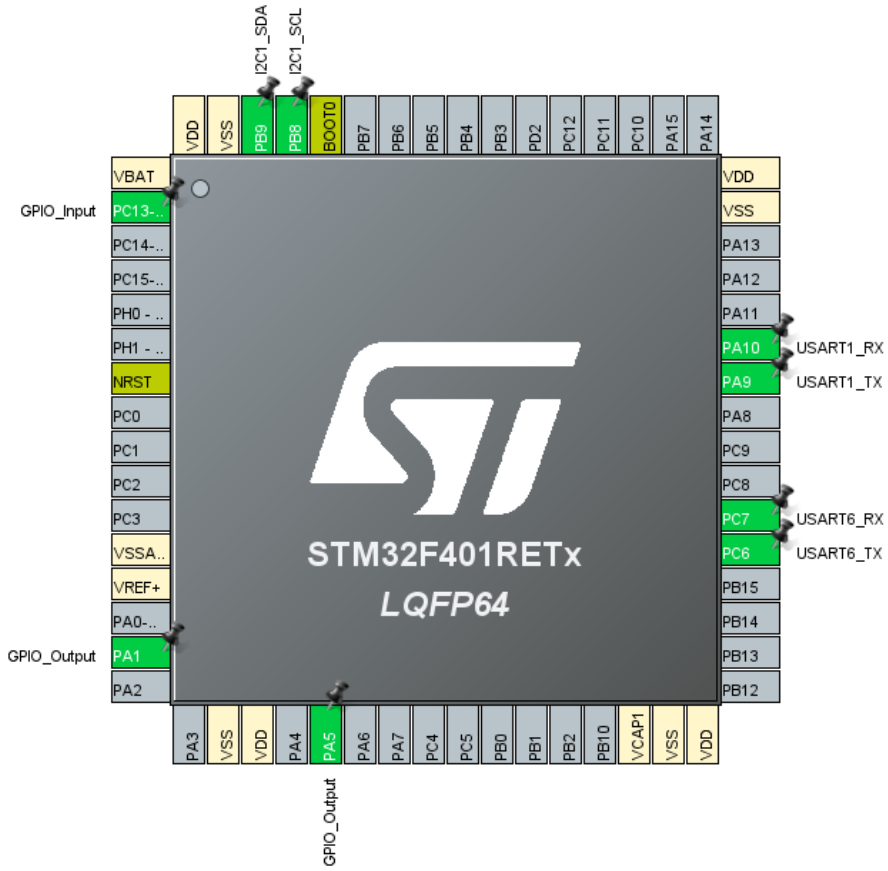
GIOCATORE 1-2: MACCHINA A STATI 2



MASTER: CONFIGURAZIONI I

La scheda **Master** è configurata come **centro di controllo** del sistema di gioco e richiede le seguenti periferiche:

A. **PA1 (LED VERDE):** segnale luminoso per indicare il momento GO!
B. **PA5 (LED Diagnostico):** verifica funzionamento sistema
C. **PC13 (USER BUTTON)**: input per avviare una nuova partita.
D. **USART1**: canale dedicato per la comunicazione bidirezionale col giocatore I.
E. **USART6**: canale dedicato per la comunicazione bidirezionale col giocatore 2.
F. **I2C1(Display OLED SSD1306):** interfaccia utente per visualizzazione stato gioco, countdown, risultati e menu.
G. **TIM2**: Gestione timing del countdown casuale e timestamping per sequenze temporali del gioco.



MASTER: CONFIGURAZIONI 2

USART1 Mode and Configuration

Mode

Mode: Asynchronous

Hardware Flow Control (RS232): Disable

Configuration

Reset Configuration

NVIC Settings | DMA Settings | GPIO Settings

Parameter Settings | User Constants

Configure the below parameters :

Search (Ctrl+F)

- Basic Parameters
  - Baud Rate: 115200 Bits/s
  - Word Length: 8 Bits (including Parity)
  - Parity: None
  - Stop Bits: 1
- Advanced Parameters
  - Data Direction: Receive and Transmit
  - Over Sampling: 16 Samples

I2C1 Mode and Configuration

Mode

I2C: I2C

Configuration

Reset Configuration

NVIC Settings | DMA Settings | GPIO Settings

Parameter Settings | User Constants

Configure the below parameters :

Search (Ctrl+F)

- Master Features
  - I2C Speed Mode: Standard Mode
  - I2C Clock Speed (Hz): 100000
- Slave Features
  - Clock No Stretch Mode: Disabled
  - Primary Address Length sele... 7-bit
  - Dual Address Acknowledged: Disabled
  - Primary slave address: 0
  - General Call address detectio. Disabled

GPIO Mode and Configuration

Mode

Configuration

Group By Peripherals

GPIO | I2C | USART

Search Signals

Search (Ctrl+F)

Show only Modified Pins

Pin ...	Signal o...	GPIO o...	GPIO m...	GPIO P...	Maximu...	User La...	Modified
PA1	n/a	Low	Output ...	No pull-...	Low		☐
PA5	n/a	Low	Output ...	No pull-...	Low		☐
PC13-...	n/a	n/a	Input m...	No pull-...	n/a		☐

TIM2 Mode and Configuration

Mode

Slave Mode: Disable

Trigger Source: Disable

Configuration

Reset Configuration

NVIC Settings | DMA Settings

Parameter Settings | User Constants

Configure the below parameters :

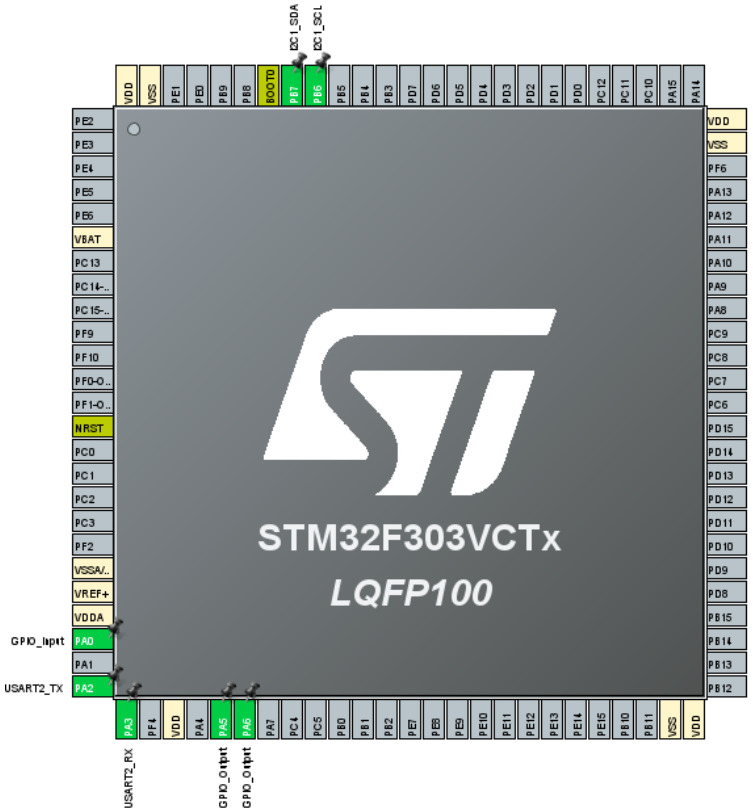
Search (Ctrl+F)

- Counter Settings
  - Prescaler (PSC - 16 bits valu... 0
  - Counter Mode: Up
  - Counter Period (AutoReload ... 4294967295
  - Internal Clock Division (CKD): No Division
  - auto-reload preload: Disable
- Trigger Output (TRGO) Parameters
  - Master/Slave Mode (MSM bit): Disable (Trigger input effect not delaye.
  - Trigger Event Selection: Reset (UG bit from TIMx_EGR)

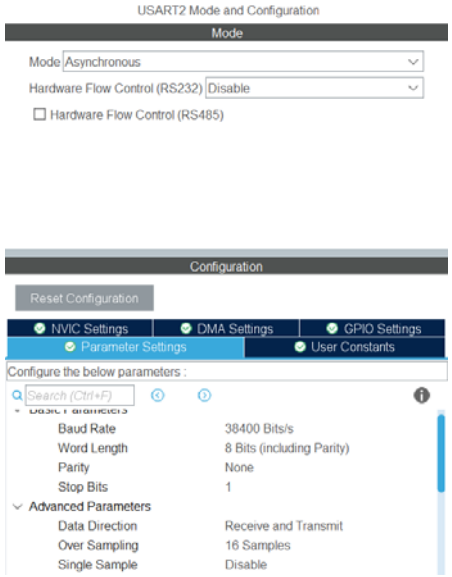
GIOCATORE I-2: CONFIGURAZIONI I

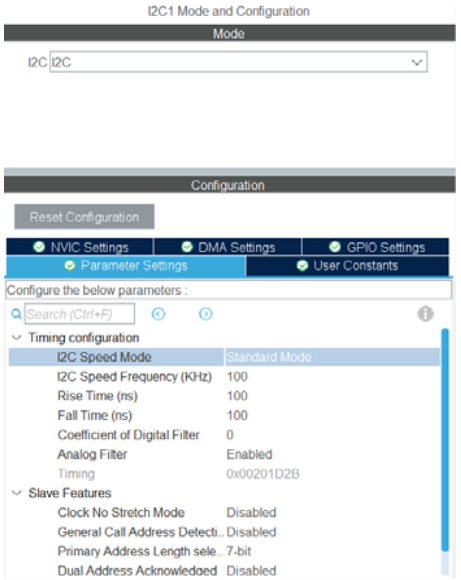
La scheda **Slave** è configurata come **Player Controller** individuale e richiede le seguenti periferiche:

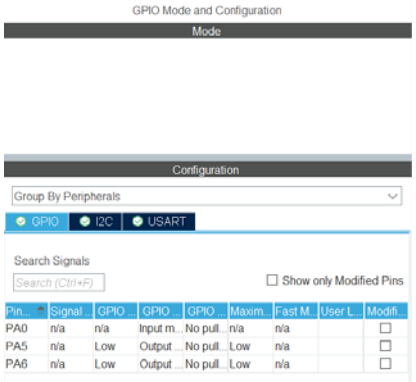
A. **PA0 (USER BUTTON)**: input del giocatore per registrare il tempo di reazione al segnale «GO!».
B. **PA5 (LED FEEDBACK)**: feedback visivo locale che si accende quando il giocatore preme il pulsante.
C. **PA6 (BUZZER)**: feedback sonoro per conferma pressione e notifica vittoria (500ms).
D. **USART2**: canale dedicato per la comunicazione bidirezionale con il Master.
E. **I2C1(Display OLED SSD1306):** interfaccia utente per visualizzazione stato gioco, istruzioni e risultato personale.
F. **TIM2**: misurazione critica del tempo di reazione con calcolo autonomo della differenza temporale tra comando «GO!» e pressione del pulsante.

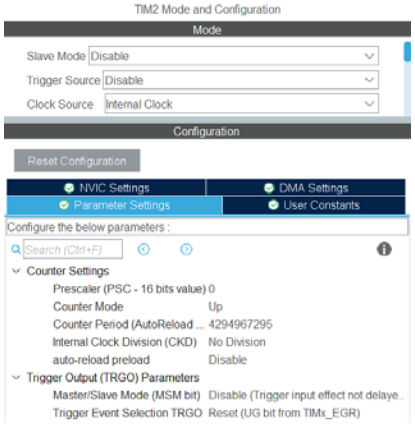


GIOCATORE I-2: CONFIGURAZIONI 2

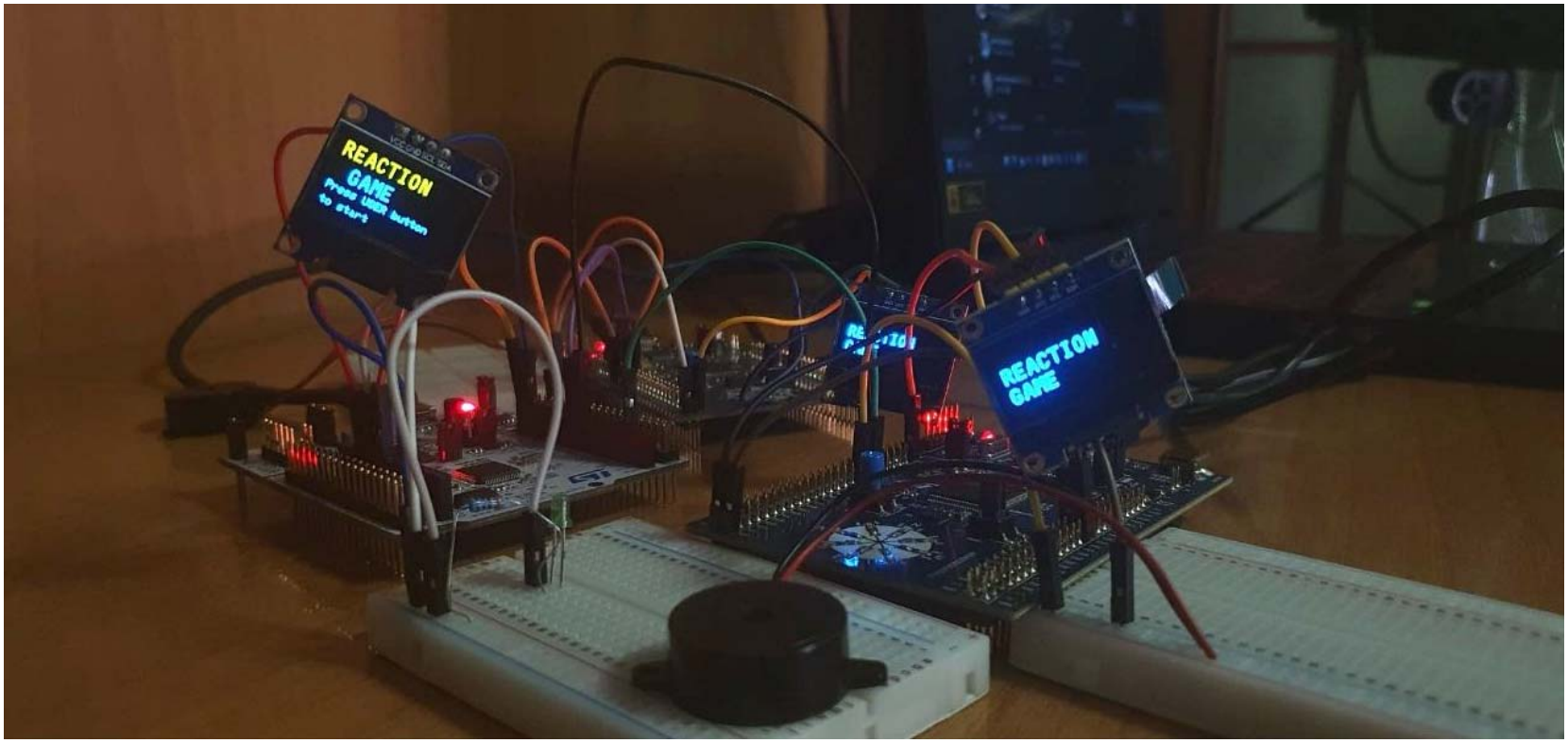








COLLEGAMENTI



MASTER: IMPLEMENTAZIONE I

```c
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
typedef enum {
    GAME_IDLE,
    GAME_WAITING,
    GAME_LED_ON,
    GAME_FINISHED,
    GAME_FALSE_START
} GameState_t;

typedef struct {
    uint32_t timestamp;
    uint8_t player_id;
    uint8_t valid;
} ReactionTime_t;
/* USER CODE END PTD */

```

Definizione struttura stati e tempo di reazione

```c
/* USER CODE BEGIN PV */
volatile GameState_t game_state = GAME_IDLE;
volatile uint32_t game_start_time = 0;
volatile uint32_t led_on_time = 0;
volatile uint32_t random_delay = 0;
```

Definizione variabili d'utilità

```c
// Inizializza display OLED
HAL_Delay(100);
ssd1306_Init();
ssd1306_Fill(Black);
ssd1306_SetCursor(0, 0);
ssd1306_WriteString("REACTION", Font_11x18, White);     // ✅ CORRETTO
ssd1306_SetCursor(15, 20);
ssd1306_WriteString("GAME", Font_11x18, White);         // ✅ NUOVA RIGA
ssd1306_SetCursor(0, 40);
ssd1306_WriteString("Press USER button", Font_6x8, White);
ssd1306_SetCursor(0, 55);
ssd1306_WriteString("to start", Font_6x8, White);
ssd1306_UpdateScreen();
```

Inizializzazione display OLED

MASTER: IMPLENTAZIONE 2

```c
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
  uint32_t current_time = GetCurrentTime();

  switch(game_state) {
      case GAME_IDLE:
          // Aspetta pressione user button per iniziare
          if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
              StartNewGame();
              HAL_Delay(300); // Debounce
          }
          break;

      case GAME_WAITING:
          // Aspetta il delay casuale
          if (current_time - game_start_time >= random_delay) {
              // Accendi LED di start
              HAL_GPIO_WritePin(GPIOA, LED_START_PIN, GPIO_PIN_SET);
              led_on_time = current_time;  // Salva il momento di accensione LED
              game_state = GAME_LED_ON;

              // Invia comando GO a entrambi i slave
              uint8_t go_cmd[] = "GO\r\n";
              HAL_UART_Transmit(&huart1, go_cmd, strlen((char*)go_cmd), 100);
              HAL_UART_Transmit(&huart6, go_cmd, strlen((char*)go_cmd), 100);

              // Aggiorna display
              ssd1306_Fill(Black);
              ssd1306_SetCursor(30, 20);
              ssd1306_WriteString("GO!", Font_11x18, White);
              ssd1306_UpdateScreen();
          }
          break;

      case GAME_LED_ON:
          // Aspetta reazioni dei giocatori o timeout
          if (player1_time.valid && player2_time.valid) {
              // Entrambi hanno reagito
              game_state = GAME_FINISHED;
          } else if (current_time - led_on_time > 5000) {
              // Timeout 5 secondi - finisce anche se solo uno ha reagito
              game_state = GAME_FINISHED;
          }
          break;

      case GAME_FINISHED:
          // Mostra risultati
          UpdateDisplay();

          // Aspetta 5 secondi poi torna al menu
          HAL_Delay(5000);
          game_state = GAME_IDLE;

          // Reset display
          ssd1306_Fill(Black);
          ssd1306_SetCursor(0, 0);
          ssd1306_WriteString("REACTION", Font_11x18, White);     // ✅ CORRETTO
          ssd1306_SetCursor(15, 20);
          ssd1306_WriteString("GAME", Font_11x18, White);         // ✅ NUOVA RIGA
          ssd1306_SetCursor(0, 40);
          ssd1306_WriteString("Press USER button", Font_6x8, White);
          ssd1306_SetCursor(0, 55);
          ssd1306_WriteString("to start", Font_6x8, White);
          ssd1306_UpdateScreen();
          break;

      case GAME_FALSE_START:
          // Gestisce partenza falsa
          HAL_Delay(3000);
          game_state = GAME_IDLE;
          break;
  }

  HAL_Delay(10);
}
```

Switch case: realizzazione dell'automa a stati finiti

MASTER: IMPLEMENTAZIONE 3

```c
void StartNewGame(void) {
    // Reset variabili di gioco per entrambi i giocatori
    player1_time.valid = 0;
    player1_time.timestamp = 0;
    player2_time.valid = 0;
    player2_time.timestamp = 0;

    // Spegni LED
    HAL_GPIO_WritePin(GPIOA, LED_START_PIN, GPIO_PIN_RESET);

    // Genera delay casuale
    GenerateRandomDelay();

    game_start_time = GetCurrentTime();
    game_state = GAME_WAITING;

    // Aggiorna display
    ssd1306_Fill(Black);
    ssd1306_SetCursor(0, 10);
    ssd1306_WriteString("GET", Font_11x18, White);
    ssd1306_SetCursor(0, 35);
    ssd1306_WriteString("READY...", Font_11x18, White);
    sprintf(display_buffer, "Wait %lu sec", random_delay/1000);
    ssd1306_SetCursor(0, 55);
    ssd1306_WriteString(display_buffer, Font_6x8, White);
    ssd1306_UpdateScreen();

    // Invia comando di reset a entrambi i slave
    uint8_t reset_cmd[] = "RESET\r\n";
    HAL_UART_Transmit(&huart1, reset_cmd, strlen((char*)reset_cmd), 100);
    HAL_UART_Transmit(&huart6, reset_cmd, strlen((char*)reset_cmd), 100);
}
```

Funzione StartNewGame, richiamata in GAME_IDLE

```c
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART1) {
        // Player 1
        uart1_rx_index++;
        if (uart1_rx_index >= UART_RX_BUFFER_SIZE - 1 ||
            uart1_rx_buffer[uart1_rx_index-1] == '\n') {
            uart1_rx_buffer[uart1_rx_index] = '\0';
            ProcessUARTMessage(uart1_rx_buffer, uart1_rx_index, 1);
            uart1_rx_index = 0;
        }
        HAL_UART_Receive_IT(&huart1, &uart1_rx_buffer[uart1_rx_index], 1);
    }

    if (huart->Instance == USART6) {
        // Player 2
        uart6_rx_index++;
        if (uart6_rx_index >= UART_RX_BUFFER_SIZE - 1 ||
            uart6_rx_buffer[uart6_rx_index-1] == '\n') {
            uart6_rx_buffer[uart6_rx_index] = '\0';
            ProcessUARTMessage(uart6_rx_buffer, uart6_rx_index, 2);
            uart6_rx_index = 0;
        }
        HAL_UART_Receive_IT(&huart6, &uart6_rx_buffer[uart6_rx_index], 1);
    }
}
```

ISR Ricezione UART

MASTER: IMPLEMENTAZIONE 4

```c
void ProcessUARTMessage(uint8_t *buffer, uint8_t len, uint8_t player_id) {
    // Cerca timestamp nel messaggio formato "TIME:12345"
    char *time_str = strstr((char*)buffer, "TIME:");
    if (time_str != NULL) {
        uint32_t reaction_time_ms = atoi(time_str + 5);  // Tempo in millisecondi dallo Slave

        // Verifica se è partenza falsa
        if (game_state != GAME_LED_ON) {
            game_state = GAME_FALSE_START;
            ssd1306_Fill(Black);
            ssd1306_SetCursor(0, 10);
            ssd1306_WriteString("FALSE", Font_11x18, White);
            ssd1306_SetCursor(0, 35);
            sprintf(display_buffer, "START P%d!", player_id);
            ssd1306_WriteString(display_buffer, Font_11x18, White);
            ssd1306_UpdateScreen();
            return;
        }

        // Il tempo di reazione è già corretto dallo Slave
        if (player_id == 1 && !player1_time.valid) {
            player1_time.timestamp = reaction_time_ms;  // Usa direttamente il tempo dello Slave
            player1_time.valid = 1;
        } else if (player_id == 2 && !player2_time.valid) {
            player2_time.timestamp = reaction_time_ms;  // Usa direttamente il tempo dello Slave
            player2_time.valid = 1;
        }
    }
}
```

Funzione di processing messaggio UART

```c
void UpdateDisplay(void) {
    // Spegni LED
    HAL_GPIO_WritePin(GPIOA, LED_START_PIN, GPIO_PIN_RESET);

    ssd1306_Fill(Black);

    if (player1_time.valid || player2_time.valid) {
        ssd1306_SetCursor(0, 0);
        ssd1306_WriteString("RESULTS:", Font_7x10, White);

        // Player 1
        if (player1_time.valid) {
            uint32_t seconds = player1_time.timestamp / 1000;
            uint32_t milliseconds = player1_time.timestamp % 1000;
            sprintf(display_buffer, "P1: %lu.%03lu sec", seconds, milliseconds);
            ssd1306_SetCursor(0, 15);
            ssd1306_WriteString(display_buffer, Font_7x10, White);
        } else {
            ssd1306_SetCursor(0, 15);
            ssd1306_WriteString("P1: NO PRESS", Font_7x10, White);
        }

        // Player 2
        if (player2_time.valid) {
            uint32_t seconds = player2_time.timestamp / 1000;
            uint32_t milliseconds = player2_time.timestamp % 1000;
            sprintf(display_buffer, "P2: %lu.%03lu sec", seconds, milliseconds);
            ssd1306_SetCursor(0, 30);
            ssd1306_WriteString(display_buffer, Font_7x10, White);
        } else {
            ssd1306_SetCursor(0, 30);
            ssd1306_WriteString("P2: NO PRESS", Font_7x10, White);
        }

        // ✅ NUOVO: Determina vincitore e invia comandi agli slave
        if (player1_time.valid && player2_time.valid) {
            if (player1_time.timestamp < player2_time.timestamp) {
                // Player 1 vince
                ssd1306_SetCursor(0, 50);
                ssd1306_WriteString("WINNER: P1!", Font_7x10, White);

```

Funzione visualizzazione risultati game Parte 1

MASTER: IMPLEMENTAZIONE 5

```c
            // Invia risultati agli slave
            uint8_t win_cmd[] = "WIN\r\n";
            uint8_t lose_cmd[] = "LOSE\r\n";
            HAL_UART_Transmit(&huart1, win_cmd, strlen((char*)win_cmd), 100);   // Slave 1 vince
            HAL_UART_Transmit(&huart6, lose_cmd, strlen((char*)lose_cmd), 100); // Slave 2 perde

        } else if (player2_time.timestamp < player1_time.timestamp) {
            // Player 2 vince
            ssd1306_SetCursor(0, 50);
            ssd1306_WriteString("WINNER: P2!", Font_7x10, White);

            // Invia risultati agli slave
            uint8_t win_cmd[] = "WIN\r\n";
            uint8_t lose_cmd[] = "LOSE\r\n";
            HAL_UART_Transmit(&huart1, lose_cmd, strlen((char*)lose_cmd), 100); // Slave 1 perde
            HAL_UART_Transmit(&huart6, win_cmd, strlen((char*)win_cmd), 100);   // Slave 2 vince

        } else {
            // Pareggio - entrambi perdono tecnicamente
            ssd1306_SetCursor(0, 50);
            ssd1306_WriteString("TIE!", Font_7x10, White);

            // In caso di pareggio, entrambi "perdono"
            uint8_t lose_cmd[] = "LOSE\r\n";
            HAL_UART_Transmit(&huart1, lose_cmd, strlen((char*)lose_cmd), 100);
            HAL_UART_Transmit(&huart6, lose_cmd, strlen((char*)lose_cmd), 100);
        }
    } else if (player1_time.valid) {
        // Solo Player 1 ha premuto
        ssd1306_SetCursor(0, 50);
        ssd1306_WriteString("WINNER: P1!", Font_7x10, White);

        uint8_t win_cmd[] = "WIN\r\n";
        uint8_t lose_cmd[] = "LOSE\r\n";
        HAL_UART_Transmit(&huart1, win_cmd, strlen((char*)win_cmd), 100);   // Slave 1 vince
        HAL_UART_Transmit(&huart6, lose_cmd, strlen((char*)lose_cmd), 100); // Slave 2 perde

    } else if (player2_time.valid) {
        // Solo Player 2 ha premuto
        ssd1306_SetCursor(0, 50);
        ssd1306_WriteString("WINNER: P2!", Font_7x10, White);

        uint8_t win_cmd[] = "WIN\r\n";
        uint8_t lose_cmd[] = "LOSE\r\n";
        HAL_UART_Transmit(&huart1, lose_cmd, strlen((char*)lose_cmd), 100); // Slave 1 perde
        HAL_UART_Transmit(&huart6, win_cmd, strlen((char*)win_cmd), 100);   // Slave 2 vince
    }
} else {
    // Nessuno ha premuto
    ssd1306_SetCursor(0, 20);
    ssd1306_WriteString("NO RESPONSE", Font_11x18, White);
    ssd1306_SetCursor(0, 45);
    ssd1306_WriteString("FROM BOTH!", Font_7x10, White);

    // Entrambi perdono
    uint8_t lose_cmd[] = "LOSE\r\n";
    HAL_UART_Transmit(&huart1, lose_cmd, strlen((char*)lose_cmd), 100);
    HAL_UART_Transmit(&huart6, lose_cmd, strlen((char*)lose_cmd), 100);
}

ssd1306_UpdateScreen();
}
```

Funzione visualizzazione risultati game – Parti 2 e 3

GIOCATORE 1-2: IMPLEMENTAZIONE 1

```c
typedef enum {
    SLAVE_IDLE,
    SLAVE_READY,
    SLAVE_BUTTON_PRESSED,
    SLAVE_SHOW_RESULT
} SlaveState_t;
```

Definizione struttura stati

```c
/* USER CODE BEGIN PV */
volatile SlaveState_t slave_state = SLAVE_IDLE;
volatile uint8_t button_pressed = 0;
volatile uint32_t button_press_time = 0;
volatile uint8_t game_active = 0;
volatile uint32_t buzzer_start_time = 0;
volatile uint8_t buzzer_active = 0;
volatile uint32_t go_received_time = 0;
volatile uint8_t show_result = 0;
volatile uint8_t player_won = 0;  // 1 = vittoria, 0 = sconfitta
volatile uint32_t result_start_time = 0;  // ✅ NUOVO: tempo inizio visualizzazione risultato
```

Definizione variabili d'utilità

```c
    // Gestione buzzer con timeout
    if (buzzer_active && (current_time - buzzer_start_time >= BUZZER_DURATION)) {
        DeactivateBuzzer();
    }
```

Funzione per disabilitare automaticamente il Buzzer

GIOCATORE I-2: IMPLEMENTAZIONE 2

```c
    switch(slave_state) {
        case SLAVE_IDLE:
            // Aspetta comando di reset/start dal master
            break;

        case SLAVE_READY:
            // Aspetta pressione del user button
            if (HAL_GPIO_ReadPin(GPIOA, USER_BUTTON_PIN) == GPIO_PIN_SET && !button_pressed) {
                button_pressed = 1;
                button_press_time = current_time;
                slave_state = SLAVE_BUTTON_PRESSED;

                // Attiva LED e buzzer
                HAL_GPIO_WritePin(GPIOA, LED_PIN, GPIO_PIN_SET);
                ActivateBuzzer();

                // Mostra messaggio di pressione
                DisplayPressedScreen();

                // Invia tempo di reazione al master
                SendReactionTime(button_press_time);
            }
            break;

        case SLAVE_BUTTON_PRESSED:
            // Aspetta rilascio del pulsante e comando di reset
            if (HAL_GPIO_ReadPin(GPIOA, USER_BUTTON_PIN) == GPIO_PIN_RESET) {
                button_pressed = 0;
            }
            break;

        case SLAVE_SHOW_RESULT:
            // Mostra risultato per 5 secondi poi torna a idle
            // ✅ CORREZIONE: Usa result_start_time invece di HAL_Delay
            if (current_time - result_start_time >= 5000) {
                slave_state = SLAVE_IDLE;
                DisplayIdleScreen();
            }
            break;
    }

    HAL_Delay(5); // Piccolo delay per ridurre carico CPU
  }
  /* USER CODE END 3 */
}
```

Switch case: implementazione automa a stati finiti

GIOCATORE I-2: IMPLEMENTAZIONE 3

```c
// Callback UART per ricezione comandi dal master
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART2) {
        uart_rx_index++;
        if (uart_rx_index >= UART_RX_BUFFER_SIZE - 1 ||
            uart_rx_buffer[uart_rx_index-1] == '\n') {
            uart_rx_buffer[uart_rx_index] = '\0';
            ProcessUARTCommand(uart_rx_buffer, uart_rx_index);
            uart_rx_index = 0;
        }
        HAL_UART_Receive_IT(&huart2, &uart_rx_buffer[uart_rx_index], 1);
    }
}
```

ISR ricezione UART

```c
void ProcessUARTCommand(uint8_t *buffer, uint8_t len) {
    // Comando di reset per iniziare nuovo gioco
    if (strstr((char*)buffer, "RESET") != NULL) {
        ResetSlave();
        slave_state = SLAVE_READY;
        game_active = 1;

        // Mostra schermo di gioco pronto
        DisplayReadyScreen();
    }

    // Comando GO - il LED si è acceso
    if (strstr((char*)buffer, "GO") != NULL) {
        go_received_time = GetCurrentTime();

        // Mostra schermo GO
        DisplayGoScreen();
    }

    // Comando WIN - questo slave ha vinto
    if (strstr((char*)buffer, "WIN") != NULL) {
        ShowGameResult(1);  // 1 = vittoria
        slave_state = SLAVE_SHOW_RESULT;
    }

    // Comando LOSE - questo slave ha perso
    if (strstr((char*)buffer, "LOSE") != NULL) {
        ShowGameResult(0);  // 0 = sconfitta
        slave_state = SLAVE_SHOW_RESULT;
    }
}
```

Funzione di processing messaggio UART

GIOCATORE I-2: IMPLEMENTAZIONE 4

```c
void SendReactionTime(uint32_t button_timestamp) {
    // Calcola tempo di reazione dal comando GO
    uint32_t reaction_time = button_timestamp - go_received_time;
    sprintf(tx_buffer, "TIME:%lu\r\n", reaction_time);
    HAL_UART_Transmit(&huart2, (uint8_t*)tx_buffer, strlen(tx_buffer), 100);
}
```

Funzione trasmissione tempi di reazione

```c
// Funzione per mostrare risultato del gioco
void ShowGameResult(uint8_t won) {
    ssd1306_Fill(Black);

    if (won) {
        // Vittoria
        ssd1306_SetCursor(15, 15);
        ssd1306_WriteString("HAI", Font_11x18, White);
        ssd1306_SetCursor(10, 35);
        ssd1306_WriteString("VINTO!", Font_11x18, White);

        // ✅ CORREZIONE: Buzzer breve per vittoria (già gestito dal timeout nel main loop)
        ActivateBuzzer();
    } else {
        // Sconfitta
        ssd1306_SetCursor(15, 15);
        ssd1306_WriteString("HAI", Font_11x18, White);
        ssd1306_SetCursor(10, 35);
        ssd1306_WriteString("PERSO!", Font_11x18, White);
    }

    ssd1306_UpdateScreen();
    show_result = 1;
    player_won = won;
    result_start_time = GetCurrentTime(); // ✅ NUOVO: salva momento inizio risultato
}
```

Funzione visualizzazione risultato game

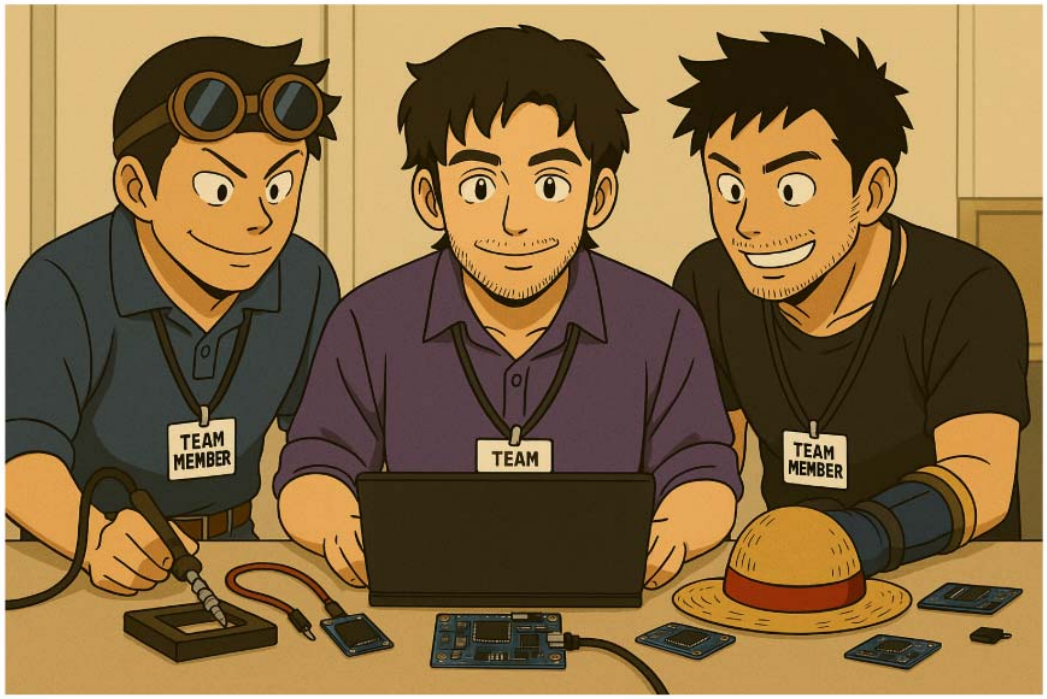
```c
void ResetSlave(void) {
    // Reset variabili di stato
    button_pressed = 0;
    button_press_time = 0;
    game_active = 0;
    go_received_time = 0;
    show_result = 0;
    player_won = 0;
    result_start_time = 0; // ✅ NUOVO: reset tempo risultato

    // Spegni LED e buzzer
    HAL_GPIO_WritePin(GPIOA, LED_PIN, GPIO_PIN_RESET);
    DeactivateBuzzer();

    slave_state = SLAVE_IDLE;
}
```

Funzione per il reset dello stato del giocatore

TESTING E CONSIDERAZIONI FINALI

Nel video allegato alla presentazione sono state verificate tutte le principali **condizioni di gioco**, attraverso una serie di **test** che includono:

A. Partita con risposta corretta da parte di entrambi i giocatori.
B. Partita senza alcuna risposta da parte dei giocatori.
C. Partita con risposta esclusiva del Giocatore 1.
D. Partita con risposta esclusiva del Giocatore 2.
E. Caso di falsa partenza.

Inoltre, è stata introdotta una **prima implementazione sperimentale** per la gestione di un nuovo scenario di gioco: il **pareggio**. In questa modalità, qualora i due giocatori reagiscano simultaneamente, il sistema assegna la sconfitta a entrambi.

IL TEAM